search...

如梦山河千帆舞，夕朝日耀照吾影。不为凡器锢我情，青春本因江湖游！

XichenOC + 关注

园龄：8个月 粉丝：7 关注：0

收藏 闪存 小组 博问

算法学习笔记 rope

2025-01-21 21:02 阅读: 475 评论: 1 推荐: 1

神级STL结构-rope大法（学习笔记）

简介：

他是一个 STL 中自带的一种数据结构，是 **pb_ds（Policy-Based Data Structures）库** 的一个分支，由于他的底层是可持久化平衡树——红黑树，或块状链表。所以他的操作的复杂度几乎约等于 $O(logn)$ 或 $\sqrt{n}$ 。它可支持操作较多，完全可以理解成加强版的 $vector$。

最为抽象的是，它可以几乎以 $O(1)$ 的复杂度复制一份，且空间消耗不算太大，这导致他可以代替实现繁琐的可持久化平衡树、可持久化线段树（主席树）、可持久化并查集等等。

其可以以很小的代码量和调试难度骗得较高得分，且 $noip$ 可以使用。

使用方法：

准备：

对与 $rope$ 的使用必须要要调用其库和名名空间名。（因为它不包含在万能头中，且不是 std 命名）

```
#include<ext/rope>
using namespace __gnu_cxx;
```

定义：

其定义方式与其他 STL 的定义方式相同，如 `rope<int>a` 。若要定义一个字符 $rope$ 可以 `rope<char>c` 或 `crope c` ，其类似与一个字符串。

操作：

字符串rope（crope）

首先，我们定义了一个 `crope a;`

- `a.push_back(x)` ：在 `a` 的末尾添加字符 `c` ；
- `a.insert(p,x)` ：在 `a` 的下标 `p` 后面添加 `x` ；
- 或 `a.insert(p,s,n)` ：将字符串 `s` 的前 `n` 位插入 `a` 的下标 `p` 处；
- `a.erase(p,x)` ：从 `a` 的下标 `p` 开始删除 `x` 个元素；
- `a.replace(p,s)` ：从 `a` 的下标 `p` 开始换成字符串 `s` ，若 `a` 的位数不够则补足；
- `a.copy(p,n,s)` ：从 `a` 的下标 `p` 开始的 `n` 个字符换成字符串 `s` ，若位数不够则补足；

- a.substr(p,x) ：从 a 的下标 p 开始截取 x 个元素；
- a[x] 或 a.at(x) 访问下标为 x 的元素；
- a.append(s,p,n) ：把字符串 s 中从下标 p 开始的 n 个字符连接到 a 的结尾，如没有参数 n 则把字符串 s 中下标 p 后的所有字符连接到 a 的结尾，如参数 p 也没有则把整个字符串 s 连接到 a 的结尾；

以上， s 均为字符串（ string 或 char[] ）类型， c 为字符（ char ）类型， n ， p ， x 均为整型。

数组rope（rope）

和字符串 rope 的操作差不多，只是把上文中的字符串参数换成了数组。

数组 rope 可以用 a.append(x) 来在末尾插入一个数 x ，当然也可以用类似于上文的方法把一个数组连接到末尾，这里不再赘述。

数组 rope 也支持 substr 操作，用于截取其中的一段数。

其他常用操作：

与其他普通的 STL 一样支持：

- size() ：返回大小；
- empty() ：返回是否为空；
- begin() ：返回首指针；
- end() ：返回末尾指针后一位；
- clear() ：清空该 $rope$；

可持久操作：

可持久操作主要分为两种，朴素版和指针版，由于复杂度差不多，且指针版可能不太好理解 ~~可能是我菜~~，这里主要将朴素版

我们定义两个 $rope$ 一个为 now 和 $past$ 分别表示当前正在维护的，和之前的版本。对于每一次操作结束后都可以将

`past[++cnt]=now` （没错直接等于就 $O(1)$ 复制了）其中 cnt 为版本数。这样需要调用时就直接找到历史版本查询即可。

例题：

P1383 高级打字机

P6166 [IOI2012] scrivener

两道题几乎一模一样的。

我们直接插入，储存版本，输出就可以，用 $rope$ 十分简单，与 $vector$ 无二。

```
#include<bits/stdc++.h>
#include<ext/rope>
using namespace std;
using namespace __gnu_cxx;
const int N=1e6+10;
rope<char>now,past[N];
int main(){
    int n;
    scanf("%d",&n);
    int cnt;
    while(n--){
        char op;
        cin>>op;
        if(op=='T'){
            char k;
            cin>>k;
            now.push_back(k);
            past[++cnt]=now;
        }
        else if(op=='U'){
            int k;
            scanf("%d",&k);
            now=past[cnt-k];
            past[++cnt]=now;
        }
        else if(op=='Q'){
            int k;
            scanf("%d",&k);
            printf("%c\n",now[k]);
        }
```

```
    }
}
```

优缺点：

优点：

写起来简单，学起来简单，调起来简单

相比各种可持久化数据结构，$rope$ 可谓简短而明了，而且写起来不费劲也不费时，因此对于学不懂/没有来得及学/没有时间写可持久化数据结构的人来说，$rope$ 简直是天降之物，救赎苍生；

缺点：

- 运行较慢：

> 对于它单次操作复杂度为 $O(\sqrt{n})$，在特定情况下较快，但对于根据题目变换而对应的专门的数据结构，其效率是远远不如的。

- 空间较大：

> 由于是完全复制，他的空间远远不如专门改良过后的可持久化数据结构。所以要小心使用

- 可以维护的东西较少

> 对于有些特定的操作就难以维护,不如手写的可自定义化强

练习：

1.自带版本控制功能的IDE Version Controlled IDE **题解**
2.P3850 [TJOI2007] 书架 **题解**

上一篇 P1486 [NOI2004] 郁闷的出纳员
下一篇 P3850 [TJOI2007] 书架

本文作者：XichenOC
本文链接：
https://www.cnblogs.com/XichenOC/p/18684418
版权声明：本作品采用知识共享署名-非商业性使用-禁止演绎 2.5 中国大陆许可协议进行许可。

关注我 收藏该文 1 0

Posted @ 2025-01-21 21:02 XichenOC 阅读(475) 评论(1) 收藏 举报

博客园 © 2004-2025